

GRADUAÇÃO

 FIAP

# MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



# AULA 11

Atualizando e excluindo documentos

Completando o ciclo dos dados no Cloud Firestore

# O QUE ACONTECE DEPOIS DE SALVAR?

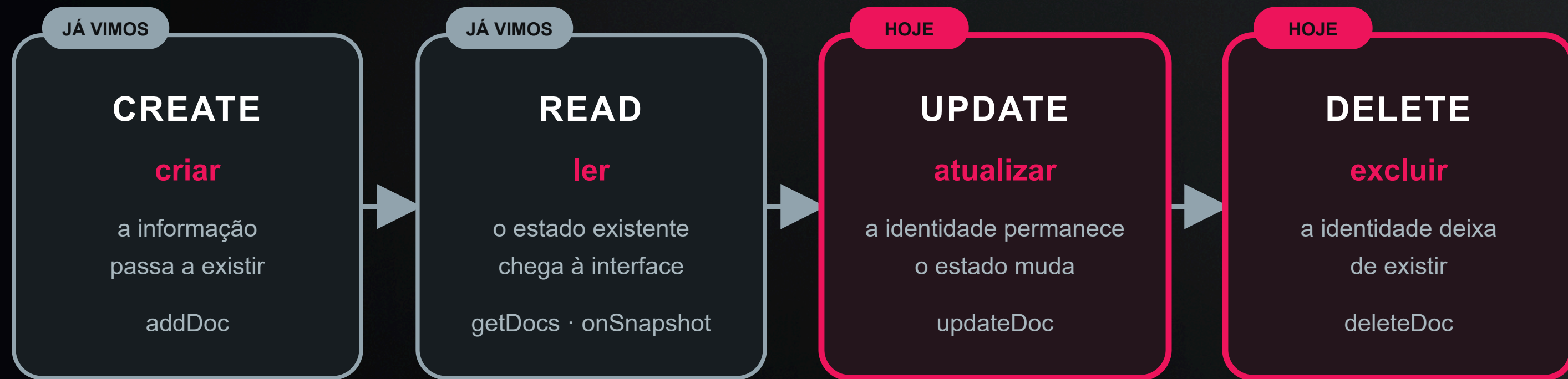
Nas últimas aulas, aprendemos a:

- criar documentos no Cloud Firestore;
- consultar os dados para exibí-los no aplicativo;
- acompanhar mudanças em tempo real.

Durante o uso do aplicativo, uma informação salva pode precisar de uma mudança ou deixar de ser necessária.

Nesta aula, vamos completar esse fluxo com **atualização** e **exclusão** de documentos existentes.

# O CICLO DOS DADOS NO APLICATIVO



**CRUD** reúne quatro intenções: criar, ler, atualizar e excluir.  
Hoje, o foco está nas duas últimas.

# OBJETIVOS

- usar o identificador da leitura para apontar para o documento correto;
- atualizar um campo com `updateDoc`, preservando a identidade;
- excluir um documento com `deleteDoc`, após a confirmação da pessoa;
- distinguir a mudança exibida na lista da confirmação da escrita no servidor.

# UMA ATIVIDADE DE ESTUDOS JÁ CADASTRADA

Considere um aplicativo que organiza atividades de estudo. Ele já criou o documento `/tasks/task-42`:

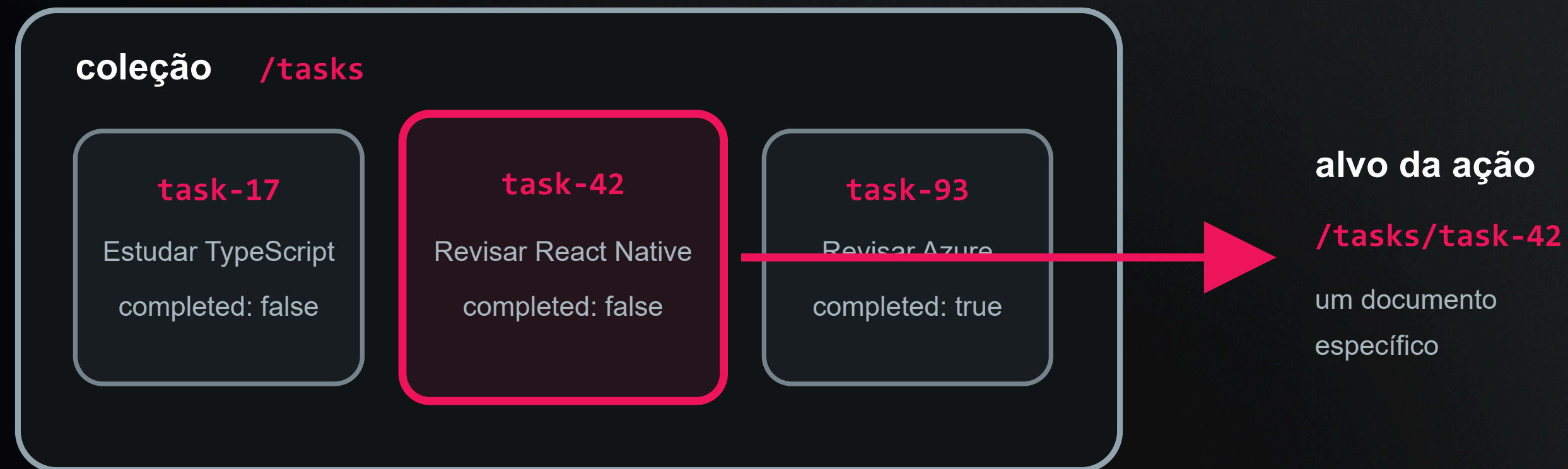
| Campo                  | Valor                               |
|------------------------|-------------------------------------|
| <code>title</code>     | <code>"Revisar React Native"</code> |
| <code>subject</code>   | <code>"Mobile"</code>               |
| <code>completed</code> | <code>false</code>                  |

Agora a pessoa pode:

- **concluir:** manter a atividade e mudar `completed` para `true`;
- **remove:** fazer a atividade deixar de existir.

Essas ações exemplificam UPDATE e DELETE sobre o mesmo documento.

# A OPERAÇÃO PRECISA DE UM ALVO ESPECÍFICO



Uma coleção indica **onde estão várias atividades**. Atualizar ou excluir exige indicar **qual atividade** sofrerá a ação.

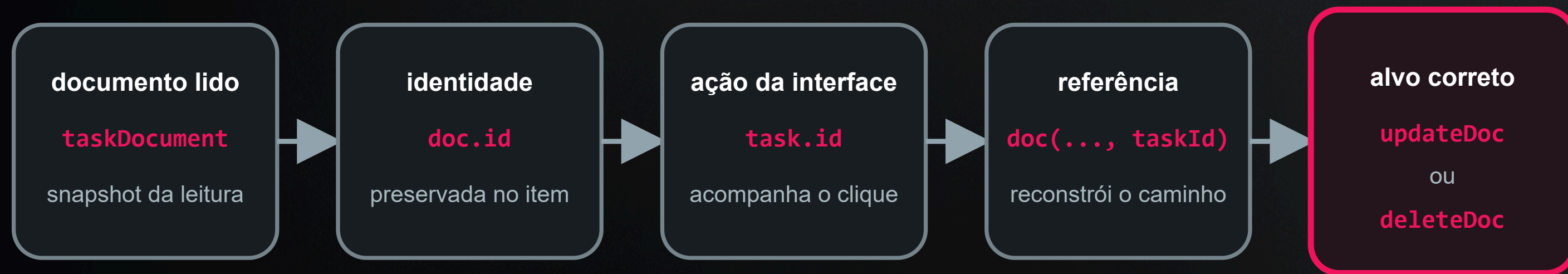
# TÍTULO E POSIÇÃO NÃO SÃO IDENTIDADE

| Informação do item       | Característica                              |
|--------------------------|---------------------------------------------|
| título                   | pode se repetir ou mudar                    |
| posição na lista         | muda com ordenação, inserção ou remoção     |
| identificador na coleção | forma o caminho junto com o nome da coleção |

```
const task = {  
  ...taskDocument.data(),  
  id: taskDocument.id,  
};
```

`taskDocument.id` vem do snapshot. A aplicação o combina com os campos para levar a identidade do documento até a ação na interface.

# O IDENTIFICADOR ACOMPANHA TODA A AÇÃO



## A AÇÃO USA TASK.ID PARA RECONSTRUIR O CAMINHO

Mesmo que o item mude de posição, a referência continua apontando para o mesmo documento.

# COLEÇÃO E DOCUMENTO REPRESENTAM ALVOS DIFERENTES

```
import { collection, doc } from "firebase/firestore";

const tasksCollection = collection(db, "tasks");
const taskReference = doc(db, "tasks", taskId);
```

| Referência          | Caminho de exemplo | Representa               |
|---------------------|--------------------|--------------------------|
| CollectionReference | /tasks             | o conjunto de atividades |
| DocumentReference   | /tasks/task-42     | uma atividade específica |

`db` é a instância do Firestore já inicializada no projeto. Construir a referência apenas aponta para o alvo, sem executar uma leitura ou escrita.

# ATUALIZAR MANTÉM A IDENTIDADE E MUDA O ESTADO

ANTES

/tasks/task-42

```
title:    "Revisar React Native"
subject:  "Mobile"
completed: false
```

UPDATE

mesmo ID

DEPOIS

/tasks/task-42

```
title:    "Revisar React Native"
subject:  "Mobile"
completed: true
```

mesma identidade · novo estado

UPDATE produz um novo estado para **a mesma identidade**, no mesmo caminho.

# A ATUALIZAÇÃO PODE MODIFICAR SOMENTE O NECESSÁRIO

Para concluir a atividade, a mudança proposta contém um único campo:

```
const changes = {
  completed: true,
};
```

| Elemento      | Resultado          |
|---------------|--------------------|
| identificador | preservado         |
| caminho       | preservado         |
| completed     | alterado para true |
| title         | preservado         |
| subject       | preservado         |

Atualização parcial modifica os campos informados e preserva os demais.

# updateDoc TRADUZ A MUDANÇA PARA O SDK

```
import { doc, updateDoc } from "firebase/firestore";

async function completeTask(taskId: string): Promise<void> {
  const taskReference = doc(db, "tasks", taskId);

  await updateDoc(taskReference, {
    completed: true,
  });
}
```

- `taskId` e a coleção formam o caminho do documento escolhido;
  - o objeto informa somente o campo que deve mudar;
  - `await` aguarda a confirmação do servidor ou propaga a falha.
- `updateDoc` exige que o documento exista. Se o alvo não existir, a operação falha.

# EXCLUIR REMOVE O DOCUMENTO INTEIRO

ANTES

/tasks/task-42

title: "Revisar React Native"

subject: "Mobile"

completed: true

DELETE

DEPOIS

/tasks/task-42

o documento não existe mais  
e deixa de participar das leituras

não sobra um documento "vazio"

DELETE remove o documento e seus campos. Ele deixa de participar das leituras da coleção.

Documentos em subcoleções precisam de remoção separada.

# `deleteDoc` TRADUZ A REMOÇÃO PARA O SDK

```
import { deleteDoc, doc } from "firebase/firestore";

async function removeTask(taskId: string): Promise<void> {
  const taskReference = doc(db, "tasks", taskId);

  await deleteDoc(taskReference);
}
```

Como esta aula não implementa lixeira ou restauração, a interface confirma a consequência **antes** de chamar `removeTask`.

O `await` aguarda a confirmação da exclusão pelo servidor.

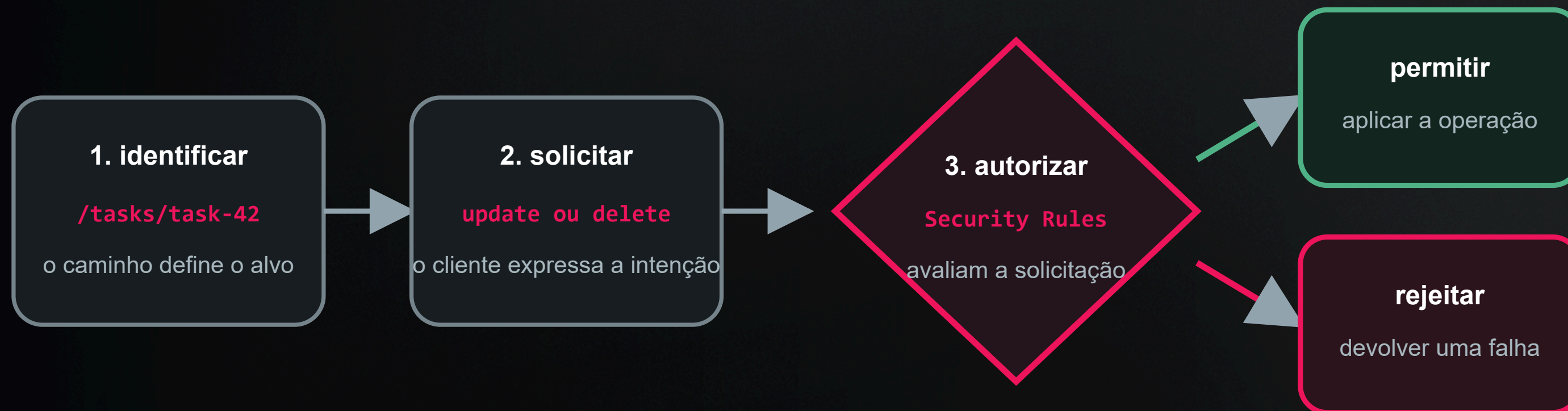
# ESCONDER O CARD NÃO EXCLUI O DOCUMENTO

| Ação                                      | Evidência                            |
|-------------------------------------------|--------------------------------------|
| filtrar o item do estado local            | o card desaparece da interface atual |
| aguardar <code>deleteDoc</code> sem falha | o servidor confirmou a exclusão      |
| conferir o documento no Firebase Console  | o documento já não existe no banco   |

O desaparecimento do card, sozinho, não comprova a exclusão remota. O listener também pode refletir uma escrita local ainda pendente.

## A CONFIRMAÇÃO VEM DA PROMISE DA OPERAÇÃO

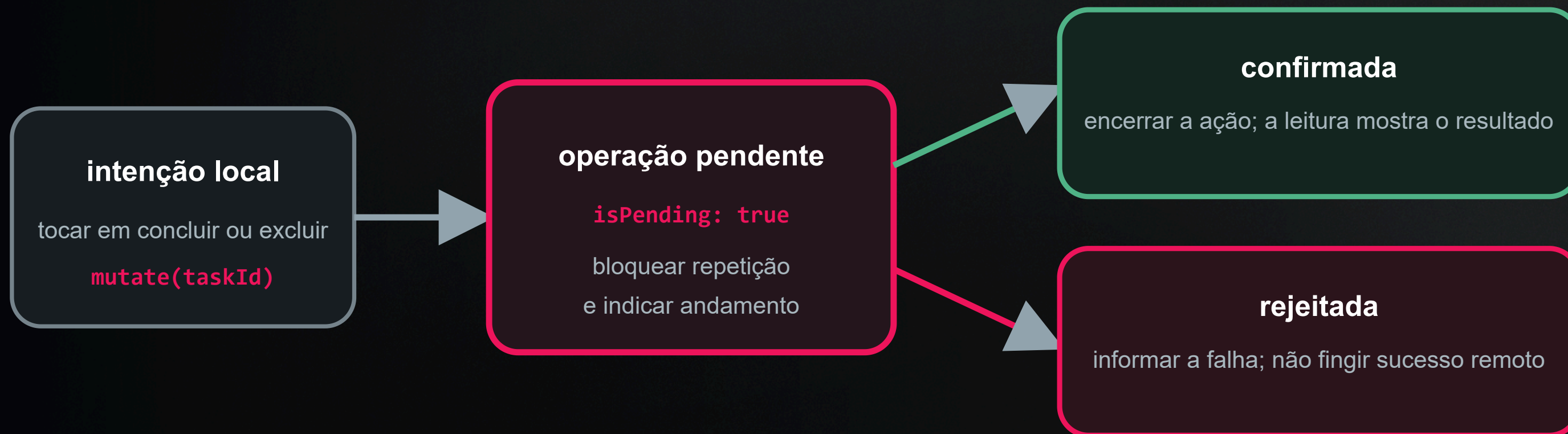
# IDENTIFICAR O CAMINHO NÃO CONCEDE PERMISSÃO



O caminho define **qual** documento é o alvo. As Security Rules decidem **se** a pessoa autenticada pode realizar a operação.

Esconder um botão não protege o dado no serviço remoto.

# ATUALIZAR E EXCLUIR SÃO OPERAÇÕES ASSÍNCRONAS



- UPDATE e DELETE são mutations: tentam alterar estado remoto e podem falhar.
- Sem conexão com o servidor, a Promise pode continuar pendente.

# SINCRONIZAÇÃO DA LISTA



Com `onSnapshot`, a lista pode mudar **antes** de a Promise resolver. A mutation continua pendente até a confirmação do servidor.

# QUATRO NECESSIDADES, QUATRO OPERAÇÕES

| Necessidade                             | Operação | Alvo principal      | API associada        |
|-----------------------------------------|----------|---------------------|----------------------|
| registrar uma informação nova           | CREATE   | coleção             | addDoc               |
| conhecer informações existentes         | READ     | coleção ou consulta | getDocs / onSnapshot |
| mudar parte de uma informação existente | UPDATE   | documento           | updateDoc            |
| retirar uma informação existente        | DELETE   | documento           | deleteDoc            |

Para UPDATE e DELETE, a aplicação preserva o `doc.id`, reconstrói o caminho do documento e representa pendência, sucesso e falha.

# DEMONSTRAÇÃO COM O MESMO DOCUMENTO

| Demonstração do professor                             | Construção com a turma                    |
|-------------------------------------------------------|-------------------------------------------|
| localizar <code>task.id</code> no item                | nomear o documento escolhido              |
| formar <code>doc(db, "tasks", task.id)</code>         | dizer qual campo muda                     |
| executar <code>completeTask(task.id)</code>           | confirmar que o documento permanece       |
| confirmar e executar <code>removeTask(task.id)</code> | indicar como a lista receberá o resultado |

Confira o mesmo caminho no Console após cada operação. A conclusão preserva o documento e os outros campos. A exclusão remove o documento.



## APLICATIVO “PACEUP”

Favoritando e excluindo check-ins com Cloud Firestore

[Acesse o Codelab](#)

## O QUE VOCÊ VAI APRENDER?

- favoritar um check-in com `updateDoc` ;
- confirmar e excluir um documento com `deleteDoc` ;
- representar pendência e falha com `useMutation` ;
- acompanhar o resultado pelo listener existente.

# REFERÊNCIAS

- Cloud Firestore: atualização de documentos
- Cloud Firestore: exclusão de documentos
- Firebase JavaScript API: Firestore
- Listeners e alterações locais

# LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026